hdoc day 50 by trishil chauhan

C Programming - Majority Element Using Pointers

Aim: Read a variable-length integer array and use a separate function **majority_element()** to count each distinct value and print the element occurring strictly more than floor(size / 2) times, or -1 when none exists.

Complete C program

```
#include <stdio.h>

void majority_element(const int *nums, int size) {
    int i, j, count, alreadyCounted;

    for (i = 0; i < size; i++) {
        alreadyCounted = 0;
        for (j = 0; j < i; j++) {
            if (*(nums + j) == *(nums + i)) {
                alreadyCounted = 1;
                break;
            }
        }
        if (alreadyCounted)
            continue;

        count = 0;
        for (j = 0; j < size; j++) {
            if (*(nums + j) == *(nums + i))
                count++;
        }
        printf("Element %d occurs %d time(s).\n",
               *(nums + i), count);
        if (count > size / 2) {
            printf("Majority element: %d\n", *(nums + i));
            return;
        }
    }
    printf("Majority element: -1\n");
}

int main(void) {
    int size, i;
    printf("Enter size of array: ");
    if (scanf("%d", &size) != 1 || size <= 0) {
        printf("Invalid array size.\n");
        return 1;
    }
    int nums[size];
    printf("Enter %d integers: ", size);
    for (i = 0; i < size; i++) {
        if (scanf("%d", &nums[i]) != 1) {
            printf("Invalid input.\n");
            return 1;
        }
    }
    majority_element(nums, size);
    return 0;
}
```

Sample output - majority exists

```
Enter size of array: 7
Enter 7 integers: 2 2 1 2 3 2 2
Element 2 occurs 5 time(s).
Majority element: 2
```

Sample output - no majority

```
Enter size of array: 6
Enter 6 integers: 1 1 2 2 3 3
Element 1 occurs 2 time(s).
Element 2 occurs 2 time(s).
Element 3 occurs 2 time(s).
Majority element: -1
```

Explanation

The array is passed as an address to the function and accessed through pointer arithmetic. For each previously unseen value, the program counts all occurrences. A majority exists only when its count is greater than size / 2 (integer division). The function prints -1 if no such value is found. This straightforward approach takes O(n squared) time and O(1) extra space.